

```

restart;
a_val := 10;
b_val := 2;
with(LinearAlgebra):
with(plots):
with(plottools):
scalarProd := ( $\varphi, \Psi$ )  $\rightarrow$  int(( $\varphi \cdot \Psi \cdot t^{a\_val} + \text{diff}(\varphi, t) \cdot \text{diff}(\Psi, t) \cdot t^{b\_val}$ ), t =
-1..1): n := 11;
pol := [seq( $t^k, k = 0..n$ )]: matgram := Matrix(n+1, n+1, (i, j)  $\rightarrow$  scalarProd(pol[i], pol[j])):
gram := proc(l) local res, i, j, max, aux;
max := numelems(l);
res := [l[1]];
aux := 0;
for i from 2 to max do for j from 1 to i-1 do aux := aux +  $\frac{\text{scalarProd}(l[i], \text{res}[j])}{\text{scalarProd}(\text{res}[j], \text{res}[j])}$ .
res[j];
end do;
res := [op(res), l[i] - aux];
aux := 0;
end do;
return res;
end proc:
normalize := proc(p) local normFactor;
normFactor := sqrt(scalarProd(p, p));
return simplify(p / normFactor);
end proc:
ortoPol := gram(pol): normPol := [seq(normalize(ortoPol[i]), i = 1..n+1)]: TnormPol :=
[seq(simplify(t · normPol[i]), i = 1..n+1)]:
extractCoeffs := proc(polinomios, vars) local coeffs, p, i; coeffs := []; for p in polinomios do coeffs :=
[op(coeffs), [subs(t = 0, p), seq(coeff(p, t, i-1), i = 2..nops(vars))]];
end do; return coeffs; end proc:
coef := extractCoeffs(TnormPol, pol):
coefN := extractCoeffs(normPol, pol): D_mat := Matrix(n, n, (i, j)  $\rightarrow$  scalarProd(t·
normPol[j], normPol[i])):
D_mat_numeric := evalf(D_mat):
Pn := ortoPol[n+1]:
P_n := simplify(Determinant(IdentityMatrix(n) * 't' - D_mat)):
P_n_roots := map(evalf, [solve(P_n = 0, t)]):
nops(P_n_roots):
eigVals := evalf(LinearAlgebra[Eigenvalues](D_mat, output = list));
nops(eigVals):
results := [];
points := [];
eig := [];
for i to nops(P_n_roots) do points := [op(points), [ℜ(P_n_roots[i]), ℑ(P_n_roots[i])]];

```

```

end do:
for i to nops(eigVals) do eig := [op(eig), [ℜ(eigVals[i]), ℑ(eigVals[i])]];
end do:
eig_3d := [seq([ℜ(eigVals[i]), ℑ(eigVals[i]), 0], i = 1 .. nops(eigVals))]:
plot3d_eig := pointplot3d(eig_3d, symbol = solidcircle, symbolsize = 5, color =
blue, labels = ["Re", "Im", "Y''"], axes = boxed):
max_roots := []:
for k from 0 to n - 1 do D_k := D_mat[1..k + 1, 1..k + 1]:
eigVals_k := Eigenvalues(D_k, output = list):
if eigVals_k ≠ [] then eigVals_k_numeric := evalf(eigVals_k):
abs_vals := map(abs, eigVals_k_numeric):
max_index := max[index](abs_vals):
max_eig := eigVals_k_numeric[max_index]:
max_roots := [op(max_roots), [ℜ(max_eig), ℑ(max_eig)]]:
end if:
end do:
k_roots := max_roots:
# k_roots; colorList := [red, blue, green, orange, purple, brown, pink, cyan, magenta]:
plotList := []:
discList := []:
zeroList := []:
legendList := []:
layer := 1:
circunferencia := display(circle([0, 0], sqrt(3)), color = black, linestyle = dash, thickness =
1
):
pts := points:
pp := pointplot(pts, symbol = solidcircle, symbolsize = 10, color = colorList[1], style =
patchnogrid):
pe := pointplot(eig, symbol = solidcircle, symbolsize = 5, color = colorList[2], style =
patchnogrid):
k_r := pointplot(k_roots, symbol = circle, symbolsize = 18, color = colorList[4], style =
patchnogrid):
plotP_n := plot(P_n, t = -1..1, color = colorList[3], thickness = 1, labels =
["t'", "P_{n}(t)"], title = "Polinomio ortonormal de grado n", axes = boxed):
plotList := [op(plotList), pp, pe, k_r, circunferencia]:
legendList := [op(legendList), cat("a=", a_val, ", b=", b_val)]:
finalPlot := display(plotList, title = "Autovalores de D, Pbsol; \n\ y raíces de Pbsol; \n\", labels =
["Re", "Im"], axes = boxed, size = [1300, 600], scaling = constrained):
print(legendList);
finalPlot;
P_n_curve := plot3d([t, 0, P_n], t = -1..1, style = line, color = green, thickness =
2):
circunferencia_3d := spacecurve([sqrt(3)*cos(theta), sqrt(3)*sin(theta), 0], theta =
0..2 * Pi, color = black, linestyle = dash
):

```

```

final_3d_plot := display(plot3d_eig, P_n_curve, circunferencia_3d, title = "Autovalores, raíces de Pbsol; n, y
[45, 45, 0]
):
final_3d_plot;
display(plotP_n):

```

```

restart;
a_val := 2;
b_val := 10;
with(LinearAlgebra):
with(plots):
with(plottools):
scalarProd := (φ, Ψ) → int((φ · Ψ · ta_val + diff(φ, t) · diff(Ψ, t) · tb_val), t =
-1..1): n := 11;
pol := [seq(tk, k = 0..n)]: matgram := Matrix(n+1, n+1, (i, j) → scalarProd(pol[i], pol[j]]):
gram := proc(l) local res, i, j, max, aux;
max := numelems(l);
res := [l[1]];
aux := 0;
for i from 2 to max do for j from 1 to i-1 do aux := aux +  $\frac{\text{scalarProd}(l[i], \text{res}[j])}{\text{scalarProd}(\text{res}[j], \text{res}[j])}$ .
res[j];
end do;
res := [op(res), l[i] - aux];
aux := 0;
end do;
return res;
end proc:
normalize := proc(p) local normFactor;
normFactor := sqrt(scalarProd(p, p));
return simplify(p / normFactor);
end proc:
ortoPol := gram(pol): normPol := [seq(normalize(ortoPol[i]), i = 1..n+1)]: TnormPol :=
[seq(simplify(t · normPol[i]), i = 1..n+1)]:
extractCoeffs := proc(polinomios, vars) local coeffs, p, i; coeffs := []; for p in polinomios do coeffs :=
[op(coeffs), [subs(t = 0, p), seq(coeff(p, t, i-1), i = 2..nops(vars))]];
end do; return coeffs; end proc:
coef := extractCoeffs(TnormPol, pol):
coefN := extractCoeffs(normPol, pol): D_mat := Matrix(n, n, (i, j) → scalarProd(t·
normPol[j], normPol[i]]):
D_mat_numeric := evalf(D_mat):
Pn := ortoPol[n+1]:
P_n := simplify(Determinant(IdentityMatrix(n) * 't' - D_mat)):
P_n-roots := map(evalf, [solve(P_n = 0, t)]):

```

```

nops(P_n_roots):
eigVals := evalf(LinearAlgebra[Eigenvalues](D_mat, output = list));
nops(eigVals):
results := [];
points := [];
eig := [];
for i to nops(P_n_roots) do points := [op(points), [Re(P_n_roots[i]), Im(P_n_roots[i])]];
end do:
for i to nops(eigVals) do eig := [op(eig), [Re(eigVals[i]), Im(eigVals[i])]];
end do:
eig_3d := [seq([Re(eigVals[i]), Im(eigVals[i]), 0], i = 1 .. nops(eigVals))]:
plot3d_eig := pointplot3d(eig_3d, symbol = solidcircle, symbolsize = 5, color =
blue, labels = ["Re", "Im", "Y"], axes = boxed):
max_roots := [];
for k from 0 to n - 1 do D_k := D_mat[1..k + 1, 1..k + 1]:
eigVals_k := Eigenvalues(D_k, output = list):
if eigVals_k[1] != [] then eigVals_k_numeric := evalf(eigVals_k):
abs_vals := map(abs, eigVals_k_numeric):
max_index := max[index](abs_vals):
max_eig := eigVals_k_numeric[max_index]:
max_roots := [op(max_roots), [Re(max_eig), Im(max_eig)]];
end if:
end do:
k_roots := max_roots:
# k_roots; colorList := [red, blue, green, orange, purple, brown, pink, cyan, magenta]:
plotList := [];
discList := [];
zeroList := [];
legendList := [];
layer := 1:
circunferencia := display(circle([0, 0], sqrt(3)), color = black, linestyle = dash, thickness =
1
):
pts := points:
pp := pointplot(pts, symbol = solidcircle, symbolsize = 10, color = colorList[1], style =
patchnogrid):
pe := pointplot(eig, symbol = solidcircle, symbolsize = 5, color = colorList[2], style =
patchnogrid):
k_r := pointplot(k_roots, symbol = circle, symbolsize = 18, color = colorList[4], style =
patchnogrid):
plotP_n := plot(P_n, t = -1..1, color = colorList[3], thickness = 1, labels =
["t", "P_{n}(t)"], title = "Polinomio ortonormal de grado n", axes = boxed):
plotList := [op(plotList), pp, pe, k_r, circunferencia]:
legendList := [op(legendList), cat("a=", a_val, ", b=", b_val)]:
finalPlot := display(plotList, title = "Autovalores de D, Pbsol; \n\ y raíces de Pbsol; \n\ ", labels =
["Re", "Im"], axes = boxed, size = [1300, 600], scaling = constrained):

```

```

print(legendList);
finalPlot;
P_n_curve := plot3d([t, 0, P_n], t = -1..1, style = line, color = green, thickness =
2):
circunferencia_3d := spacecurve([sqrt(3)*cos(theta), sqrt(3)*sin(theta), 0], theta =
0..2 * Pi, color = black, linestyle = dash
):
final_3d_plot := display(plot3d_eig, P_n_curve, circunferencia_3d, title = "Autovalores, raíces de Pbsol; n, y
[45, 45, 0]
):
final_3d_plot;
display(plotP_n):

```

```

restart;
a_val := 2;
b_val := 10;
with(LinearAlgebra):
with(plots):
with(plottools):
scalarProd := ( $\varphi, \Psi$ )  $\rightarrow$  int( $(\varphi \cdot \Psi \cdot t^{a\_val}), t = -2..-1$ ) + int( $(diff(\varphi, t) \cdot diff(\Psi, t) \cdot$ 
 $t^{b\_val}), t = 1..2$ ): n := 11;
pol := [seq( $t^k, k = 0..n$ )]: matgram := Matrix(n+1, n+1, (i, j)  $\rightarrow$  scalarProd(pol[i], pol[j]]):
gram := proc(l) local res, i, j, max, aux;
max := numelems(l);
res := [l[1]];
aux := 0;
for i from 2 to max do for j from 1 to i-1 do aux := aux +  $\frac{scalarProd(l[i], res[j])}{scalarProd(res[j], res[j])}$ .
res[j];
end do;
res := [op(res), l[i] - aux];
aux := 0;
end do;
return res;
end proc:
normalize := proc(p) local normFactor;
normFactor := sqrt(scalarProd(p, p));
return simplify(p / normFactor);
end proc:
ortoPol := gram(pol): normPol := [seq(normalize(ortoPol[i]), i = 1..n+1)]: TnormPol :=
[seq(simplify( $t \cdot normPol[i]$ ), i = 1..n+1)]:
extractCoeffs := proc(polinomios, vars) local coeffs, p, i; coeffs := []; for p in polinomios do coeffs :=
[op(coeffs), [subs(t = 0, p), seq(coeff(p, t, i-1), i = 2..nops(vars))]];
end do; return coeffs; end proc:

```

```

coef := extractCoeffs(TnormPol, pol):
coefN := extractCoeffs(normPol, pol): D_mat := Matrix(n, n, (i, j) → scalarProd(t·
normPol[j], normPol[i])):
D_mat_numeric := evalf(D_mat):
Pn := ortoPol[n + 1]:
P_n := simplify(Determinant(IdentityMatrix(n) * 't' - D_mat)):
P_n_roots := map(evalf, [solve(P_n = 0, t)]):
nops(P_n_roots):
eigVals := evalf(LinearAlgebra[Eigenvalues](D_mat, output = list));
nops(eigVals):
results := []:
points := []:
eig := []:
for i to nops(P_n_roots) do points := [op(points), [ℜ(P_n_roots[i]), ℑ(P_n_roots[i])]];
end do:
for i to nops(eigVals) do eig := [op(eig), [ℜ(eigVals[i]), ℑ(eigVals[i])]];
end do:
eig_3d := [seq([ℜ(eigVals[i]), ℑ(eigVals[i]), 0], i = 1 .. nops(eigVals))]:
plot3d.eig := pointplot3d(eig_3d, symbol = solidcircle, symbolsize = 5, color =
blue, labels = ["Re", "Im", "Y"], axes = boxed):
max_roots := []:
for k from 0 to n - 1 do D_k := D_mat[1..k + 1, 1..k + 1]:
eigVals_k := Eigenvalues(D_k, output = list):
if eigVals_k ≠ [] then eigVals_k_numeric := evalf(eigVals_k):
abs_vals := map(abs, eigVals_k_numeric):
max_index := max[index](abs_vals):
max_eig := eigVals_k_numeric[max_index]:
max_roots := [op(max_roots), [ℜ(max_eig), ℑ(max_eig)]];
end if:
end do:
k_roots := max_roots:
# k_roots; colorList := [red, blue, green, orange, purple, brown, pink, cyan, magenta]:
plotList := []:
discList := []:
zeroList := []:
legendList := []:
layer := 1:
circunferencia := display(circle([0, 0], sqrt(3)), color = black, linestyle = dash, thickness =
1
):
pts := points:
pp := pointplot(pts, symbol = solidcircle, symbolsize = 10, color = colorList[1], style =
patchnograd):
pe := pointplot(eig, symbol = solidcircle, symbolsize = 5, color = colorList[2], style =
patchnograd):
k_r := pointplot(k_roots, symbol = circle, symbolsize = 18, color = colorList[4], style =

```

```

patchnograd):
plotP_n_1 := plot(P_n, t = -2..-1, color = colorList[3], thickness = 1, labels =
["t'", "P_{n}(t)"], title = "Polinomio ortonormal de grado n", axes = boxed):
plotP_n_2 := plot(P_n, t = 1..2, color = colorList[3], thickness = 1, labels =
["t'", "P_{n}(t)"], title = "Polinomio ortonormal de grado n", axes = boxed):
plotList := [op(plotList), pp, pe, k_r]:
legendList := [op(legendList), cat("a=", a_val, ", b=", b_val)]:
finalPlot := display(plotList, title = "Autovalores de D, Pbsol; _\n\ y raíces de Pbsol; _\n\ ", labels =
["Re", "Im"], axes = boxed, size = [1300, 600], scaling = constrained):
print(legendList);
finalPlot;
P_n_1_curve := plot3d([t, 0, P_n], t = -2..-1, style = line, color = green, thickness =
2):
P_n_2_curve := plot3d([t, 0, P_n], t = 1..2, style = line, color = green, thickness =
2):
circunferencia_3d := spacecurve([sqrt(3)*cos(theta), sqrt(3)*sin(theta), 0], theta =
0..2 * Pi, color = black, linestyle = dash
):
final_3d_plot := display(plot3d_eig, P_n_1_curve, P_n_2_curve, circunferencia_3d, title =
"Autovalores, raíces de Pbsol; _n, y Pbsol; _n(t) en 3D", orientation = [45, 45, 0]
):
final_3d_plot;

```

```

restart;
a_val := 10;
b_val := 2;
with(LinearAlgebra):
with(plots):
with(plottools):
scalarProd := (φ, Ψ) → int((φ·Ψ·ta_val), t = -2..-1) + int((diff(φ, t)·diff(Ψ, t)·
tb_val), t = 1..2): n := 11;
pol := [seq(tk, k = 0..n)]: matgram := Matrix(n+1, n+1, (i, j) → scalarProd(pol[i], pol[j])):
gram := proc(l) local res, i, j, max, aux;
max := numelems(l);
res := [l[1]];
aux := 0;
for i from 2 to max do for j from 1 to i-1 do aux := aux +  $\frac{\text{scalarProd}(l[i], \text{res}[j])}{\text{scalarProd}(\text{res}[j], \text{res}[j])}$ .
res[j];
end do;
res := [op(res), l[i] - aux];
aux := 0;
end do;
return res;
end proc;

```

```

normalize := proc(p) local normFactor;
normFactor := sqrt(scalarProd(p, p));
return simplify(p / normFactor);
end proc;

ortoPol := gram(pol): normPol := [seq(normalize(ortoPol[i]), i = 1..n+1)]: TnormPol :=
[seq(simplify(t · normPol[i]), i = 1..n+1)]:
extractCoeffs := proc(polinomios, vars) local coeffs, p, i; coeffs := []; for p in polinomios do coeffs :=
[op(coeffs), [subs(t = 0, p), seq(coeff(p, t, i - 1), i = 2..nops(vars))]];
end do; return coeffs; end proc;
coef := extractCoeffs(TnormPol, pol):
coefN := extractCoeffs(normPol, pol): D_mat := Matrix(n, n, (i, j) → scalarProd(t·
normPol[j], normPol[i])):
D_mat_numeric := evalf(D_mat):
Pn := ortoPol[n+1]:
P_n := simplify(Determinant(IdentityMatrix(n) * 't' - D_mat)):
P_n_roots := map(evalf, [solve(P_n = 0, t)]):
nops(P_n_roots):
eigVals := evalf(LinearAlgebra[Eigenvalues](D_mat, output = list));
nops(eigVals):
results := [];
points := [];
eig := [];
for i to nops(P_n_roots) do points := [op(points), [ℜ(P_n_roots[i]), ℑ(P_n_roots[i])]];
end do;
for i to nops(eigVals) do eig := [op(eig), [ℜ(eigVals[i]), ℑ(eigVals[i])]];
end do;
eig_3d := [seq([ℜ(eigVals[i]), ℑ(eigVals[i]), 0], i = 1..nops(eigVals))]:
plot3d.eig := pointplot3d(eig_3d, symbol = solidcircle, symbolsize = 5, color =
blue, labels = ["ℜ", "ℑ", "Y"], axes = boxed):
max_roots := [];
for k from 0 to n - 1 do D_k := D_mat[1..k+1, 1..k+1]:
eigVals_k := Eigenvalues(D_k, output = list):
if eigVals_k ≠ [] then eigVals_k_numeric := evalf(eigVals_k):
abs_vals := map(abs, eigVals_k_numeric):
max_index := max[index](abs_vals):
max_eig := eigVals_k_numeric[max_index]:
max_roots := [op(max_roots), [ℜ(max_eig), ℑ(max_eig)]];
end if;
end do;
k_roots := max_roots:
# k_roots; colorList := [red, blue, green, orange, purple, brown, pink, cyan, magenta]:
plotList := [];
discList := [];
zeroList := [];
legendList := [];
layer := 1:

```

```

circunferencia := display(circle([0, 0], sqrt(3)), color = black, linestyle = dash, thickness =
1
):
pts := points:
pp := pointplot(pts, symbol = solidcircle, symbolsize = 10, color = colorList[1], style =
patchnogrid):
pe := pointplot(eig, symbol = solidcircle, symbolsize = 5, color = colorList[2], style =
patchnogrid):
k_r := pointplot(k_roots, symbol = circle, symbolsize = 18, color = colorList[4], style =
patchnogrid):
plotP_n_1 := plot(P_n, t = -2..-1, color = colorList[3], thickness = 1, labels =
["t'", "P_{n}(t)"], title = "Polinomio ortonormal de grado n", axes = boxed):
plotP_n_2 := plot(P_n, t = 1..2, color = colorList[3], thickness = 1, labels =
["t'", "P_{n}(t)"], title = "Polinomio ortonormal de grado n", axes = boxed):
plotList := [op(plotList), pp, pe, k_r]:
legendList := [op(legendList), cat("a=", a_val, ", b=", b_val)]:
finalPlot := display(plotList, title = "Autovalores de D, Pbsol; \n\ y raíces de Pbsol; \n\ ", labels =
["Re", "Im"], axes = boxed, size = [1300, 600], scaling = constrained):
print(legendList);
finalPlot;
P_n_1_curve := plot3d([t, 0, P_n], t = -2..-1, style = line, color = green, thickness =
2):
P_n_2_curve := plot3d([t, 0, P_n], t = 1..2, style = line, color = green, thickness =
2):
circunferencia_3d := spacecurve([sqrt(3)*cos(theta), sqrt(3)*sin(theta), 0], theta =
0..2 * Pi, color = black, linestyle = dash
):
final_3d_plot := display(plot3d_eig, P_n_1_curve, P_n_2_curve, circunferencia_3d, title =
"Autovalores, raíces de Pbsol; n, y Pbsol; n(t) en 3D", orientation = [45, 45, 0]
):
final_3d_plot;

```